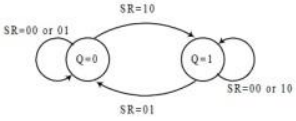
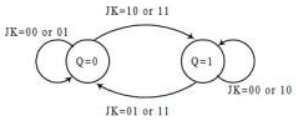
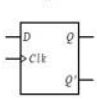
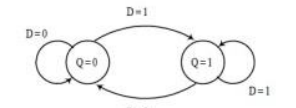
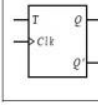
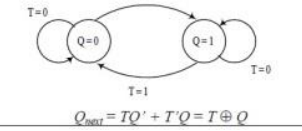
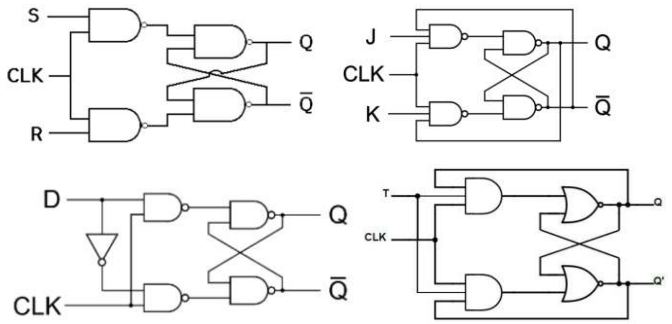
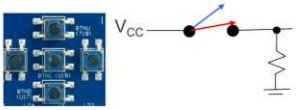
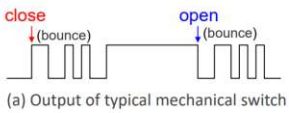
EE2026 Cheatsheet

Digital Design (National University of Singapore)

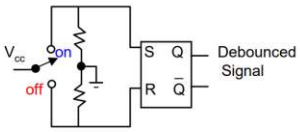
Name / Symbol	Characteristic (Truth) Table	State Diagram / Characteristic Equations	Excitation Table
SR 	$S\ R\ Q\ Q_{next}$ 0 0 0 0 0 0 1 1 0 1 0 0 0 1 1 0 1 0 0 1 1 0 1 1 1 1 0 × 1 1 1 ×	 $Q_{next} = S + R'Q$ $SR = 0$	$Q\ Q_{next}\ S\ R$ 0 0 0 × 0 1 1 0 1 0 0 1 1 1 × 0
JK 	$J\ K\ Q\ Q_{next}$ 0 0 0 0 0 0 1 1 0 1 0 0 0 1 1 0 1 0 0 1 1 0 1 1 1 1 0 1 1 1 1 0	 $Q_{next} = J'K'Q + JK' + JKQ'$ $= J'K'Q + JK'Q + JK'Q' + JKQ'$ $= K'Q(J'+J) + JQ'(K'+K)$ $= K'Q + JQ'$	$Q\ Q_{next}\ J\ K$ 0 0 0 × 0 1 1 × 1 0 × 1 1 1 × 0
D 	$D\ Q\ Q_{next}$ 0 × 0 1 × 1	 $Q_{next} = D$	$Q\ Q_{next}\ D$ 0 0 0 0 1 1 1 0 0 1 1 1
T 	$T\ Q\ Q_{next}$ 0 0 0 0 1 1 1 0 1 1 1 0	 $Q_{next} = TQ' + T'Q = T \oplus Q$	$Q\ Q_{next}\ T$ 0 0 0 0 1 1 1 0 1 1 1 0







(a) Output of typical mechanical switch

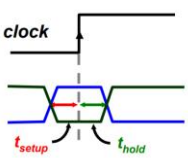


(b) Debounced switch output signal

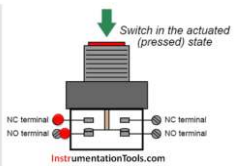
- Mechanical switches bounce before settling down which may cause problems as inputs.
- Switch debouncing is a common use of S-R FFs.

S	R	Output
0	0	Hold
0	1	Q = 0
1	0	Q = 1
1	1	Invalid

0 0 is the **rest state**



t_{setup} :	minimum time before the *active* clock edge by which FF inputs must be stable.
t_{hold}:	minimum time inputs must be stable after *active* clock edge
t_{pHL} :	time taken for FF output to change state from High to Low.
t_{pLH} :	time taken for FF output to change state from Low to High.



A FF has the following timing parameters :

$t_{pHL} = t_{pLH} = 50ns$.
t_{setup} and t_{hold} = negligible (smaller than 0.01ns)

What is the **theoretical limit** of the maximum clock frequency (in MHz) that I can apply to this FF?
Please type in only the number in MHz in the blank below :

General Feedback

The FF has a output propagation delay of 50ns (either for signal going from low-to-high, or high-to-low). This means that the output Q takes 50ns to change its value.

Thus, the maximum theoretical clock frequency that can be applied is 1/50ns = 20MHz.

In practice, we would need a clock frequency that is slightly lower than 20MHz for us to observe the output Q changing. However, this 20MHz is the theoretical limit of the operating clock frequency for the FF.

Theorems of Boolean Algebra

#	Theorem		
1	$A + A = A$	$A \cdot A = A$	Tautology Law
2	$A + 1 = 1$	$A \cdot 0 = 0$	Union Law
3	$\overline{(\bar{A})} = A$		Involution Law
4	$A + (B + C) = (A + B) + C$	$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	Associative Law
5	$\overline{A + B} = \bar{A} \cdot \bar{B}$	$\overline{A \cdot B} = \bar{A} + \bar{B}$	De Morgan's Law
6	$A + A \cdot B = A$	$A \cdot (A + B) = A$	Absorption Law
7	$A + \bar{A} \cdot B = A + B$	$A \cdot (\bar{A} + B) = A \cdot B$	
8	$AB + A\bar{B} = A$	$(A + B)(A + \bar{B}) = A$	Logical adjacency
9	$AB + \bar{A}C + BC = AB + \bar{A}C$	$(A + B)(\bar{A} + C)(B + C) = (A + B)(\bar{A} + C)$	Consensus Law

Duality (OR and AND, 0 and 1 can be interchanged)

Postulates of Boolean Algebra – cont.

4. Distributive Law
   - *i.* $x \cdot (y + z) = x \cdot y + x \cdot z$ $\quad (\cdot\ over\ +)$
   - *ii.* $x + (y \cdot z) = (x + y) \cdot (x + z)$ $\quad (+\ over\ \cdot)$

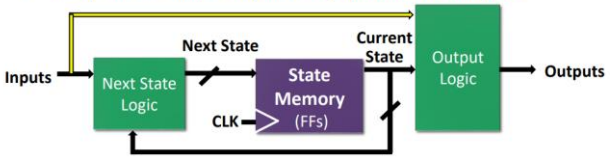
5. For every element x in the set ***B***, there exists an element $\bar{x}$ in the set ***B***, such that
   - *i.* $x + \bar{x} = 1$
   - *ii.* $x \cdot \bar{x} = 0$

   ($\bar{x}$ is called the **complement** of x)

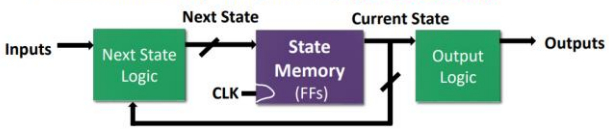
6. There exist at least two distinct elements in the set ***B***

Moore vs Mealy FSMs : Different Output Generation

- Mealy machine: *output is a function of a present state & inputs.*



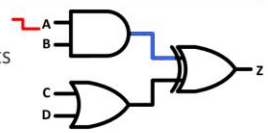
- Moore machine: *output is a function of a present state only.*



Sequential Logic Circuits?

Combinational Logic Circuits:

- Outputs depend on current inputs

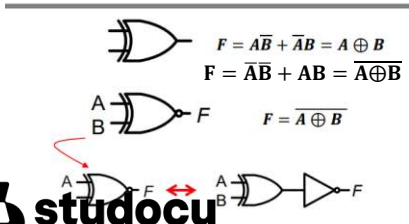


Sequential Logic Circuits:

- Outputs depend on ***current and previous*** inputs ➔ Memory!
- Requires separation of previous, current, future : **states**
- 2 Types of sequential circuits:

Synchronous	Asynchronous
Clocked: need a clock input	Unclocked
Responds to inputs at discrete time instants governed by a clock input	Responds whenever input signals change

XOR and XNOR Gates



Truth Table (XOR, XNOR):

A	B	$A \oplus B$	$\overline{A \oplus B}$
0	0	0	1
1	0	1	0
0	1	1	0
1	1	0	1

XOR

- ***F*** is TRUE if ***A ≠ B***

XNOR

- ***F*** is TRUE if ***A = B***

Operator	Description	Examples: a = 4'b1010, b=4'0000
!, ~	Logical negation, Bit-wise NOT	!a = 0, !b =1, ~a=4'b0101, ~b=4'b1111
&, |, ^	Reduction (Outputs 1-bit)	&a = 0, |a=1, ^a = 0
{__,__}	Concatenation	{b, a} = 8'b00001010
{n{___}}	Replication	{2 {a} } = 8'b10101010
*, /, %,	Multiply, *Divide, *Modulus	3 % 2 = 1, 16 % 4 = 0
+, -	Binary addition, subtraction	a + b = 4'b1010
<< , >>	Shift Zeros in Left / Right	a << 1 = 4'b0100, a >> 2 = 4'b0010
<, <=, >, >=	Logical Relative (1-bit output)	(a > b) = 1
==, !=	Logical Equality (1-bit output)	(a == b)= 0 (a != b)= 1
&, ^, |	Bit-wise AND, XOR, OR	a&b = 4'b0000 a|b = 4'b1010
&&, ||	Logical AND, OR (1-bit output)	a&&b = 0 a||b = 1
?:	Conditional Operator	<out> = <condition> ? If_ONE : if_ZERO

High ↑ Precedence ↓ Low

Blocking & Non-blocking

Verilog supports two types of assignments within **always**

= blocking assignment
- Sequential evaluation
- Immediate assignment

```
always @ (*)
begin
x = y;    1) Evaluate y, assign result to x
z = ~x;   2) Evaluate ~x, assign result to z
end
```

Behaviour	x	y	z
Initial Condition	0	0	1
y changes	0	1	1
x = y	1	1	1
z = ~x	1	1	0

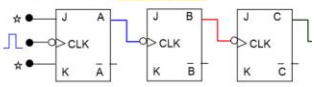
<= non-blocking asignment
- Sequential evaluation
- *Deferred* assignment

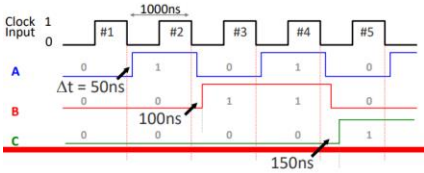
```
always @ (*)
begin
x <= y;    1) Evaluate y, defer assignment
z <= ~x;   2) Evaluate ~x, defer assignment
end        3) Assign x and z with new values
```

Behaviour	x	y	z	Deferred
Initial Condition	0	0	1	
y changes	0	1	1	
x <= y	0	1	1	x <= 1
z <= ~x	0	1	1	z <= 1
Assignment	1	1	1	

t_{pd} ➔ limiting frequency

- Ripple counters are very easy to implement
- But they have a major drawback ➔ they cannot operate beyond a limiting frequency
- Due to *propagation delays* of the FFs in the chain adding up:
  1. Clock input FF_1: t_0
  2. Clock input FF_2: $t_0 + \Delta tpd$
  3. Clock input FF_3: $t_0 + 2*\Delta tpd$
  4. Clock input FF_N: $t_0 + (n - 1) * \Delta tpd$
     ➔ the nth FF changes state at n* Δtpd after t_0





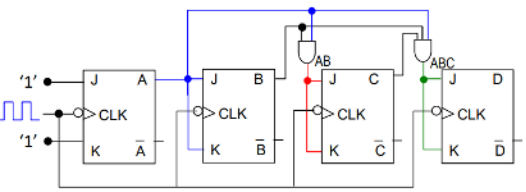
For proper operation:

$Tclock \geq n*\Delta tpd$

$fmax \leq 1 / n*\Delta tpd$

Synchronous (Parallel) Counters

Mod-16 Synchronous Parallel Counter :

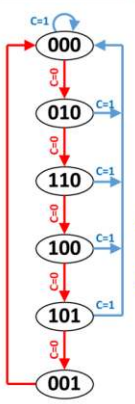


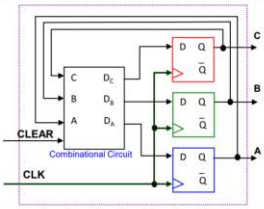
B toggles when A = 1
C toggles when AB = 1
D toggles when ABC = 1

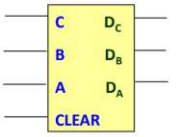
Count	D	C	B	A
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1
0	0	0	0	0
	c	o	n	t

- Since all FFs are triggered simultaneously by the clock, this counter can operate at higher frequencies.
- Total delay = Δtpd (FF) + Δtpd (AND)
- Operating frequency is irrespective of the number of FFs
- Disadvantages? ↪ t_clk >= t_pd,ff + t_pd,and

Step1: Write state diagram. **Step2**: Determine functional block diagram of counter. **Step3** : Functional block diagram of combinational circuit.







Step4 : Get TT of combinational circuit using FF excitation table. **Step5** : Realize circuit.

CLEAR	C	B	A	C+	B+	A+	D_C	D_B	D_A
0	0	0	0	0	1	0	0	1	0
0	0	0	1	0	0	0	0	0	0
0	0	1	0	1	1	0	1	1	0
0	0	1	1	X	X	X	X	X	X
0	1	0	0	1	0	1	1	0	1
0	1	0	1	0	0	1	0	0	1
0	1	1	0	1	0	0	1	0	0
0	1	1	1	X	X	X	X	X	X
1	X	X	X	0	0	0	0	0	0

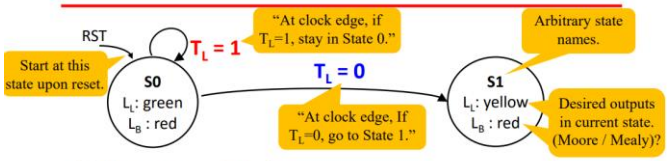
SOP for flip-flop inputs

$$D_C = \overline{CLEAR} \bullet B + \overline{CLEAR} \bullet C \bullet \overline{A}$$

$$D_B = \overline{CLEAR} \bullet \overline{C} \bullet \overline{A}$$

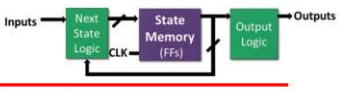
$$D_A = \overline{CLEAR} \bullet C \bullet \overline{B}$$

State Transition Diagrams



- Circles represent ***states***.
  * Each state specifies values for all outputs (Moore)
- Arcs represents ***transitions*** between states.
  * Labels ➔ input that triggers the transition.
  * Transitions take place on the active edge of the clock.
- Arc from outer space indicates initial state upon reset
- Within each state, for any combination of input values, there's exactly one applicable arc.

FSM in Verilog




```
module fsm(input clk, … , output … … );
reg __ state, nextstate;

parameter S0 = 2'b00;
parameter S1 = 2'b01;
```

`parameter` is used to define constants within a module, improving code readability.

```
always @ (*)
   case (state)
      S0  : nextstate = S1;
      S1  : nextstate = S0;
   endcase
```

Next State Combinational Logic : `(*)` code is triggered whenever any input changes ➔ combinational logic. `case` represents next state table.

```
always @ (posedge clk, posedge reset)
   if (reset) state <= S0;
   else       state <= nextstate;
```

Sequential Logic : Use `<=` to infer flip-flops. *Is this Sync or Async reset?*

```
assign y = ( state == S0 );
endmodule
```

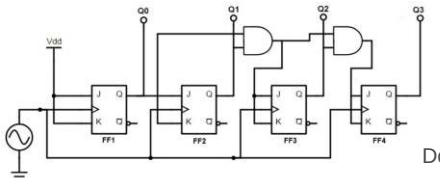
Output Logic : Use `assign` to infer combinational logic.

Equality Comparison : `a == b` evaluates to 1 if a equals b.

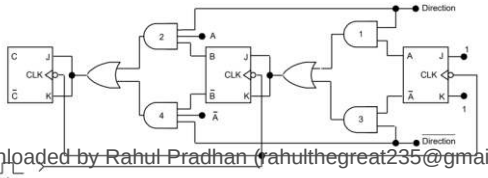

```
module counter(input clear, clk, output reg [3:0] q);

always @ (posedge clk) begin

    q <= clear ? (q - 1) : 4'b0000;

end

endmodule
```

- What counter does this code describe?
1. Positive/Negative Edge clock triggered?
2. Asynchronous / Synchronous Clear?
3. Count Up / Down Counter ?

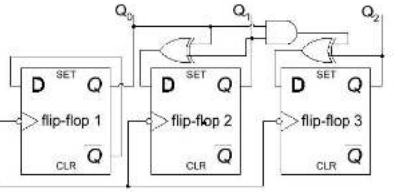
Mod-16 Count Down



Up/Down Synchronous Counter

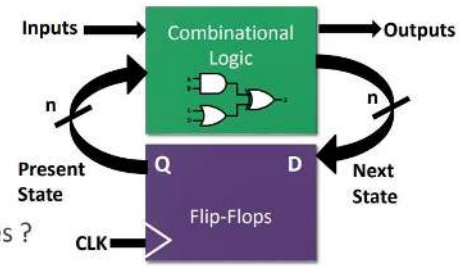


Mod-8 Count Up D-FF



What is a FSM?

- FSM : Finite State Machine
- Synchronous Machine with "states" of operation
- At each *active clock edge*, combinational logic computes **outputs** and **next state,** as a function of **inputs and present state**



- Examples ?

Structure

State Memory

- set of n FFs store current state of machine; up to 2^n states.
- FFs can be J-K or D, but D FFs simpler (1 input vs. 2 inputs for J-K FFs)

Next State Logic

- combinational circuit which decides the next state of the machine based on current state and inputs:

Next state = f (inputs, current state)

Output logic

- combinational circuit which creates the output

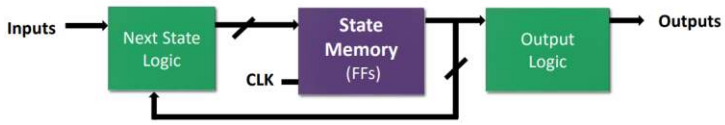
Moore : outputs depend on current state

outputs = g (current state)

Mealy : outputs depend on the current state & inputs

outputs = g (inputs, current state)

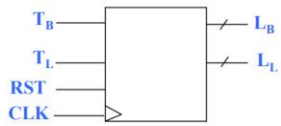
Verilog~! – Code Structure

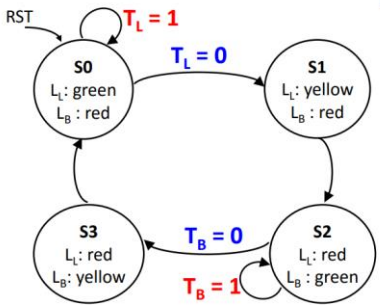


Combinational Circuit	Sequential Circuit	Combinational Circuit
`always @ (*)`	`always @ (posedge clk)`	`assign y =`

Step 1 : State Transition Diagram

- Block Diagram of Desired System
- Design ***State Transition Diagram*** to represent FSM





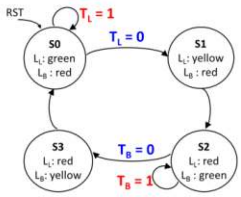
Step 2 : Next State Table

1. From STD, find the number of states
2. Number of bits / FFs required = ? to go through these states
3. State Assignment

State	S_1S_0
S0	00
S1	01
S2	10
S3	11

4. Next State Table

Current State	Inputs		Next State
S	T_L	T_B	S+
S0	**0**	X	S1
S0	**1**	X	S0
S1	X	X	S2
S2	X	**0**	S3
S2	X	**1**	S2
S3	X	X	S0

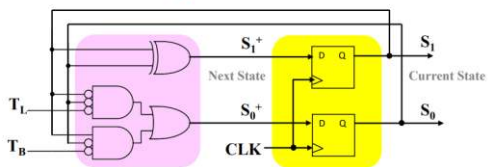


Step 2 : Next State Table

5. State Generator Circuit

Current State		Inputs		Next State	
S_1	S_0	T_L	T_B	S_1+	S_0+
0	0	**0**	X	0	1
0	0	**1**	X	0	0
0	1	X	X	1	0
1	0	X	**0**	1	1
1	0	X	**1**	1	0
1	1	X	X	0	0

$$S_1^+ = \overline{S}_1 S_0 + S_1 \overline{S}_0 \overline{T}_B + S_1 \overline{S}_0 T_B$$
$$= S_1 \oplus S_0$$
$$S_0^+ = \overline{S}_1 \overline{S}_0 \overline{T}_L + S_1 \overline{S}_0 \overline{T}_B$$



Step 3 : Output Logic

1. Output Truth Table

Output	L_1L_0
Green	00
Yellow	01
Red	10

Current State		Outputs			
S_1	S_0	L_{L1}	L_{L0}	L_{B1}	L_{B0}
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

$$L_{L1} = S_1 \quad L_{L0} = \overline{S}_1 S_0$$
$$L_{B1} = \overline{S}_1 \quad L_{B0} = S_1 S_0$$